AML White Blood Cell Classification

Learned Stain Correction

Pipeline:

```
Raw TIFF (sRGB)
  → [stain perturbation - train only]
  → RGBToLab
  → CorrectionModule (flat residual, Lab space)
  → LabToRGB
  → ImageNet Normalize
  → ConvNeXt-Tiny (pretrained, head replaced)
  → 15-class prediction
```

1. Setup & imports

```python
import os
import sys
import copy
import time
import numpy as np
import torch
import torch.nn as nn
import matplotlib.pyplot as plt

# Add project root to path
PROJECT_ROOT = "/content/drive/MyDrive/BME548_Final_Project"
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

print(f"Python  : {sys.version.split()[0]}")
print(f"PyTorch : {torch.__version__}")
print(f"CUDA    : {torch.version.cuda}")
```

```
Python  : 3.12.13
PyTorch : 2.10.0+cu128
CUDA    : 12.8
```

2. Mount Google Drive

```python
from google.colab import drive
drive.mount("/content/drive", force_remount=False)

# Verify the dataset is reachable
assert os.path.isdir(PROJECT_ROOT), f"Dataset not found at {PROJECT_ROOT}"
print(f"Dataset found at: {PROJECT_ROOT}")
```

```
Mounted at /content/drive
Dataset found at: /content/drive/MyDrive/BME548_Final_Project
```

3. Load data

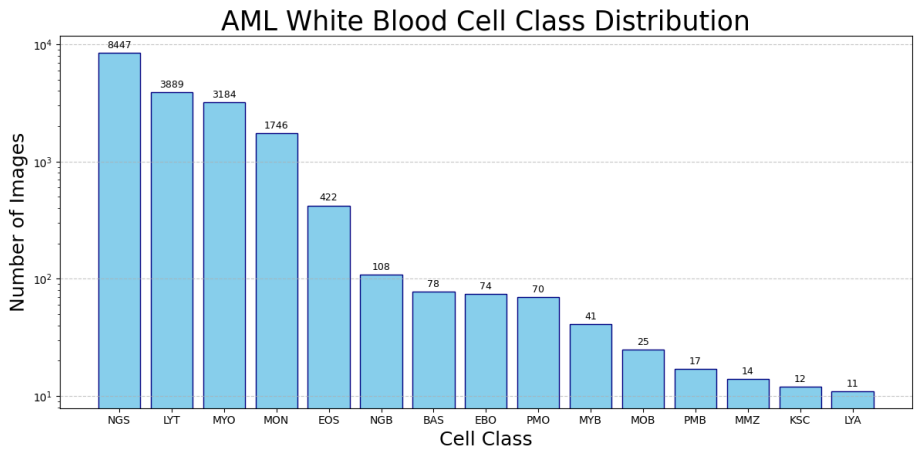
```python
import importlib
from preprocess import build_dataloaders, extract_dataset_to_local

# Hyperparameters
BATCH_SIZE = 256
SIGMA_S    = 0.05   # stain spectrum perturbation
SIGMA_C    = 0.15   # concentration perturbation
SIGMA_W    = 0.01   # white balance perturbation
ZIP_PATH   = PROJECT_ROOT + "/dataset.zip"
NUM_WORKERS = 8
print("dataset exists in: ", ZIP_PATH)
# Extract zip to local SSD
LOCAL_DATA_ROOT = extract_dataset_to_local(zip_path=ZIP_PATH)

train_loader, val_loader, test_loader, class_names, class_to_idx, abbrev_map = (
    build_dataloaders(
        data_root   = LOCAL_DATA_ROOT,
        batch_size  = BATCH_SIZE,
        num_workers = NUM_WORKERS,
        sigma_s     = SIGMA_S,
        sigma_c     = SIGMA_C,
        sigma_w     = SIGMA_W,
    )
)

NUM_CLASSES = len(class_names)
print(f"\nNumber of classes : {NUM_CLASSES}")
print(f"Train batches     : {len(train_loader)}")
print(f"Val   batches     : {len(val_loader)}")
print(f"Test  batches     : {len(test_loader)}")
print(f"num_workers       : {NUM_WORKERS}")
```

```
dataset exists in:  /content/drive/MyDrive/BME548_Final_Project/dataset.zip
Dataset already extracted at /tmp/aml_dataset/dataset  (18371 files). Skipping.
Classes (15): ['BAS', 'EBO', 'EOS', 'KSC', 'LYA', 'LYT', 'MMZ', 'MOB', 'MON', 'MYB', 'MYO',
Total images after filtering: 18138

Class distribution:
  BAS   Basophil                          78
  EBO   Erythroblast                      74
  EOS   Eosinophil                       422
```

```
  KSC   Smudge cell                        12
  LYA   Lymphocyte (atypical)              11
  LYT   Lymphocyte (typical)             3889
  MMZ   Metamyelocyte                      14
  MOB   Monoblast                          25
  MON   Monocyte                         1746
  MYB   Myelocyte                          41
  MYO   Myeloblast                       3184
  NGB   Neutrophil (band)                 108
  NGS   Neutrophil (segmented)           8447
  PMB   Promyelocyte (bilobled)            17
  PMO   Promyelocyte                       70

Split - train: 12696, val: 2721, test: 2721

Number of classes : 15
Train batches     : 50
Val   batches     : 11
Test  batches     : 11
num_workers       : 8
```

```python
import matplotlib.pyplot as plt
import numpy as np
import torch
import os
from preprocess import parse_annotations

# Define class counts from annotations
df_all = parse_annotations(os.path.join(LOCAL_DATA_ROOT, "annotations.dat"))
counts = df_all["label"].value_counts()
class_counts = torch.tensor(
    [int(counts.get(c, 1)) for c in class_names], dtype=torch.float32
)

counts_np = class_counts.cpu().numpy()
sorted_indices = np.argsort(counts_np)[::-1]
sorted_names = [class_names[i] for i in sorted_indices]
sorted_counts = counts_np[sorted_indices]

# Visualize dataset
plt.figure(figsize=(12, 6))
bars = plt.bar(sorted_names, sorted_counts, color='skyblue', edgecolor='navy')

for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + (yval * 0.05), int(yval), ha='center',
```

```
plt.title("AML White Blood Cell Class Distribution", fontsize=25)
plt.xlabel("Cell Class", fontsize=18)
plt.ylabel("Number of Images", fontsize=18)
plt.yscale('log')
plt.grid(axis='y', linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()
```




```
# Compute per-class weights for Focal Loss.
# Square-root inverse frequency compresses the weight range so rare classes
# get more attention without completely drowning out the majority classes.

from preprocess import parse_annotations
import pandas as pd

df_all = parse_annotations(os.path.join(LOCAL_DATA_ROOT, "annotations.dat"))
counts = df_all["label"].value_counts()

class_counts = torch.tensor(
    [int(counts.get(c, 1)) for c in class_names], dtype=torch.float32
)

class_weights = 1.0 / torch.sqrt(class_counts)
class_weights = class_weights / class_weights.sum() * NUM_CLASSES

print("Class weights:")
print(f"  {'Class':<6}  {'Count':>6}  {'Weight':>8}")
print("  " + "-" * 26)
for c, count, w in zip(class_names, class_counts, class_weights):
```

```
    print(f"  {c:<6}  {int(count):>6}  {w:>8.4f}")
print(f"\n  Weight ratio max/min: {class_weights.max()/class_weights.min():.1f}x")
```

```
Class weights:
  Class    Count    Weight
  --------------------------
  BAS         78    0.8414
  EBO         74    0.8638
  EOS        422    0.3617
  KSC         12    2.1451
  LYA         11    2.2405
  LYT       3889    0.1192
  MMZ         14    1.9860
  MOB         25    1.4862
  MON       1746    0.1778
  MYB         41    1.1605
  MYO       3184    0.1317
  NGB        108    0.7150
  NGS       8447    0.0809
  PMB         17    1.8022
  PMO         70    0.8881

  Weight ratio max/min: 27.7x
```

3b. Physical layer visualisation for original vs perturbed images

Shows the effect of each perturbation stage in isolation.

Each row shows one image under four conditions:

- **Original** — raw image straight from the dataset
- **__s only** — stain spectrum shift (reagent/pH variability)
- **__c only** — concentration scaling (incubation time variability)
- **__w only** — white balance shift (illuminant/microscope variability)
- **All combined** — all three perturbations together (what the model sees during training)

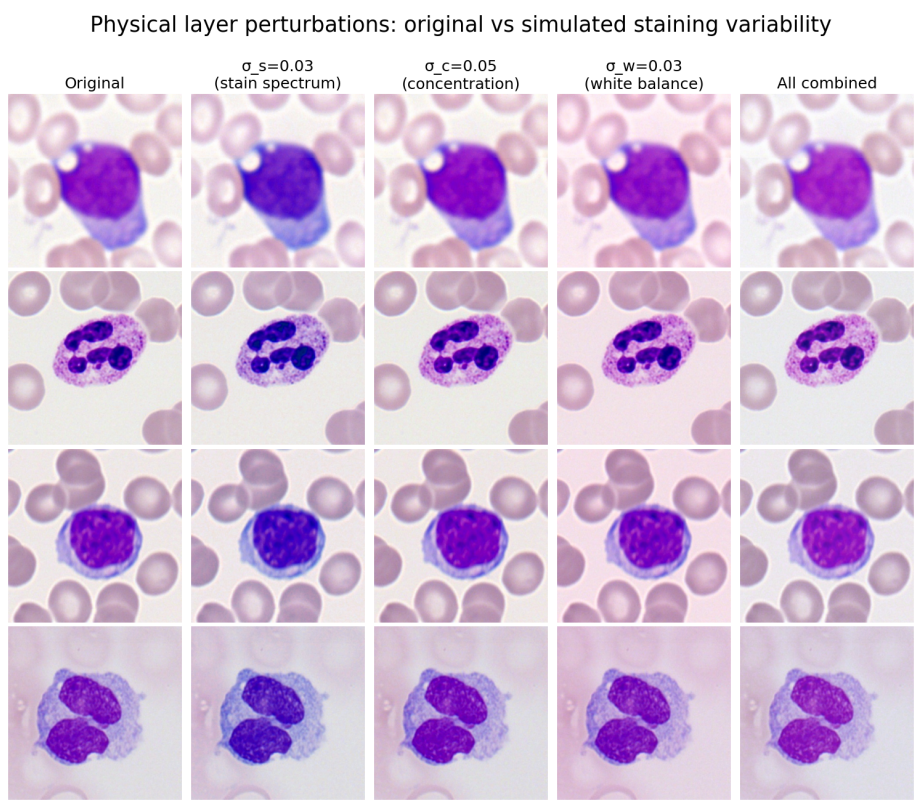
```
from preprocess import parse_annotations, resolve_image_paths, perturb_staining
from preprocess import IMAGE_FOLDER
```

```
import numpy as np
from PIL import Image
from preprocess import (
    parse_annotations, resolve_image_paths,
    perturb_staining, IMAGE_FOLDER,
)
```

```python
# Parameters
VIS_SIGMA_S  = 0.03   # realistic inter-lab stain spectrum shift
VIS_SIGMA_C  = 0.05   # realistic concentration variability
VIS_SIGMA_W  = 0.03   # realistic white balance shift
CLASSES_SHOW = ["MYO", "NGS", "LYT", "MON"]   # one image per class
N_ROWS       = len(CLASSES_SHOW)               # dynamically set rows to match classes

df_vis = parse_annotations(os.path.join(LOCAL_DATA_ROOT, "annotations.dat"))
df_vis = resolve_image_paths(df_vis, LOCAL_DATA_ROOT, IMAGE_FOLDER)

rng = np.random.default_rng(0)

col_labels = [
    "Original",
    f" _s={VIS_SIGMA_S}\n(stain spectrum)",
    f" _c={VIS_SIGMA_C}\n(concentration)",
    f" _w={VIS_SIGMA_W}\n(white balance)",
    "All combined",
]
n_cols = len(col_labels)

fig, axes = plt.subplots(
    N_ROWS, n_cols,
    figsize=(n_cols * 3, N_ROWS * 3.2),
)

for row_idx, cls_name in enumerate(CLASSES_SHOW):
    row   = df_vis[df_vis["label"] == cls_name].iloc[0]
    img   = Image.open(row["abs_path"]).convert("RGB")
    img_np = np.array(img)

    variants = [
        img_np,
        perturb_staining(img_np, sigma_s=VIS_SIGMA_S, sigma_c=0.0,          sigma_w=0.0,
        perturb_staining(img_np, sigma_s=0.0,          sigma_c=VIS_SIGMA_C, sigma_w=0.0,
        perturb_staining(img_np, sigma_s=0.0,          sigma_c=0.0,          sigma_w=VIS_SI
        perturb_staining(img_np, sigma_s=VIS_SIGMA_S, sigma_c=VIS_SIGMA_C, sigma_w=VIS_SIGMA
    ]

    for col_idx, (variant, col_label) in enumerate(zip(variants, col_labels)):
        ax = axes[row_idx, col_idx]
        ax.imshow(variant)
        ax.axis("off")

        if row_idx == 0:
            ax.set_title(col_label, fontsize=18, pad=6)
```

```
        if col_idx == 0:
            ax.set_ylabel(
                f"{cls_name}\n{abbrev_map.get(cls_name, '')}",
                fontsize=18, rotation=0, labelpad=55, va="center"
            )

fig.suptitle(
    "Physical layer perturbations: original vs simulated staining variability",
    fontsize=25, y=1.01,
)
plt.tight_layout()
save_path_pert = os.path.join(PROJECT_ROOT, "perturbation_comparison.png")
plt.savefig(save_path_pert, dpi=150, bbox_inches="tight")
print(f"Saved to {save_path_pert}")
plt.show()
```

```
Saved to /content/drive/MyDrive/BME548_Final_Project/perturbation_comparison.png
```



4. Build model

```
from models import build_model

# Hyperparameters
BASE_CHANNELS = 32   # correction module width
NUM_BLOCKS    = 3    # correction module depth
DROPOUT       = 0.3  # dropout before final linear layer

model, device = build_model(
    num_classes   = NUM_CLASSES,
    base_channels = BASE_CHANNELS,
    num_blocks    = NUM_BLOCKS,
    dropout       = DROPOUT,
    pretrained    = True,
)
```

```
GPU available: NVIDIA RTX PRO 6000 Blackwell Server Edition -
training on CUDA.
Downloading: "https://download.pytorch.org/models/convnext_tiny-983f1562.pth" to /root/.cach

100%|      | 109M/109M [00:00<00:00, 295MB/s]


Model summary:
  Correction module params :     55,872
  ConvNeXt-Tiny params     : 27,831,663
  Total params             : 27,887,535
  Trainable params         : 27,887,535
```

5. Training loop

```
from torch.optim.lr_scheduler import OneCycleLR
import os

# Training hyperparameters
NUM_EPOCHS      = 15
BATCH_SIZE      = 128
LR              = 3e-4
WEIGHT_DECAY    = 1e-4
FOCAL_GAMMA     = 2.0
LAMBDA_CONS     = 0.1    # weight for auxiliary stain consistency loss
CHECKPOINT_PATH = os.path.join(PROJECT_ROOT, "best_model.pth")

# Allow PyTorch to release cached memory back between allocations.
os.environ["PYTORCH_ALLOC_CONF"] = "expandable_segments:True"
import torch
torch.cuda.empty_cache()
```

```python
# Focal Loss
class FocalLoss(nn.Module):
    def __init__(self, alpha, gamma=2.0, reduction="mean"):
        super().__init__()
        self.register_buffer("alpha", alpha)
        self.gamma     = gamma
        self.reduction = reduction

    def forward(self, logits, targets):
        ce_loss = nn.functional.cross_entropy(
            logits, targets, reduction="none"
        )
        pt       = torch.exp(-ce_loss)
        alpha_t  = self.alpha[targets]
        focal    = alpha_t * (1.0 - pt) ** self.gamma * ce_loss
        return focal.mean() if self.reduction == "mean" else focal.sum()


criterion = FocalLoss(alpha=class_weights.to(device), gamma=FOCAL_GAMMA)

# AdamW
optimizer = torch.optim.AdamW(
    model.parameters(),
    lr           = LR,
    weight_decay = WEIGHT_DECAY,
    fused        = torch.cuda.is_available(),
)

# OneCycleLR: warmup for first 10% of steps then decay.
scheduler = OneCycleLR(
    optimizer,
    max_lr          = LR,
    steps_per_epoch = len(train_loader),
    epochs          = NUM_EPOCHS,
    pct_start       = 0.1,
)

# Mixed precision scaler
scaler = torch.amp.GradScaler("cuda")

print(f"Batch size        : {BATCH_SIZE}")
print(f"Learning rate     : {LR}")
print(f"Criterion         : FocalLoss (gamma={FOCAL_GAMMA})")
print(f"Consistency loss  : L1 in Lab space, lambda={LAMBDA_CONS}")
print(f"Optimizer         : AdamW fused={torch.cuda.is_available()}")
```

```python
print(f"Scheduler         : OneCycleLR (per-batch, warmup 10%)")
print(f"Mixed precision   : enabled (float16 activations)")
print(f"Checkpoint        : {CHECKPOINT_PATH}")
```

```
Batch size        : 128
Learning rate     : 0.0003
Criterion         : FocalLoss (gamma=2.0)
Consistency loss  : L1 in Lab space, lambda=0.1
Optimizer         : AdamW fused=True
Scheduler         : OneCycleLR (per-batch, warmup 10%)
Mixed precision   : enabled (float16 activations)
Checkpoint        : /content/drive/MyDrive/BME548_Final_Project/best_model.pth
```

5a. Run training loop

```python
from tqdm.notebook import tqdm

# Training and validation loop
train_losses, val_losses = [], []
train_accs,   val_accs   = [], []
cons_losses              = []     # track consistency loss separately
best_val_acc = 0.0
best_state   = None

for epoch in range(1, NUM_EPOCHS + 1):
    t0 = time.time()

    # Train
    model.train()
    running_loss, running_cons, correct, total = 0.0, 0.0, 0, 0

    train_bar = tqdm(
        train_loader,
        desc=f"Epoch {epoch:02d}/{NUM_EPOCHS} [train]",
        leave=False, unit="batch",
    )
    for batch in train_bar:
        imgs_perturbed, imgs_clean, labels = batch
        imgs_perturbed = imgs_perturbed.to(device)
        imgs_clean     = imgs_clean.to(device)
        labels         = labels.to(device)

        optimizer.zero_grad()

        with torch.amp.autocast("cuda"):
```

```
        # Classification loss on perturbed images
        logits   = model(imgs_perturbed)
        cls_loss = criterion(logits, labels)

        # Stain consistency loss
        cons_loss = model.compute_consistency_loss(
            imgs_perturbed, imgs_clean
        )

        loss = cls_loss + LAMBDA_CONS * cons_loss

    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)

    # Clip correction and backbone separately
    nn.utils.clip_grad_norm_(model.correction.parameters(), max_norm=1.0)
    nn.utils.clip_grad_norm_(model.backbone.parameters(),   max_norm=1.0)

    scaler.step(optimizer)
    scaler.update()
    scheduler.step()

    running_loss += cls_loss.item() * imgs_perturbed.size(0)
    running_cons += cons_loss.item() * imgs_perturbed.size(0)
    correct      += (logits.argmax(1) == labels).sum().item()
    total        += imgs_perturbed.size(0)
    train_bar.set_postfix(
        cls=f"{running_loss / total:.4f}",
        cons=f"{running_cons / total:.4f}",
        acc=f"{correct / total:.4f}",
        lr=f"{scheduler.get_last_lr()[0]:.2e}",
    )

train_loss = running_loss / total
train_cons = running_cons / total
train_acc  = correct / total

# Validate
model.eval()
val_loss_sum, val_correct, val_total = 0.0, 0, 0

val_bar = tqdm(
    val_loader,
    desc=f"Epoch {epoch:02d}/{NUM_EPOCHS} [val]  ",
    leave=False, unit="batch",
)
```

```python
    with torch.no_grad():
        for imgs, labels in val_bar:
            imgs, labels = imgs.to(device), labels.to(device)
            with torch.amp.autocast("cuda"):
                logits = model(imgs)
                loss   = criterion(logits, labels)
            val_loss_sum += loss.item() * imgs.size(0)
            val_correct  += (logits.argmax(1) == labels).sum().item()
            val_total    += imgs.size(0)
            val_bar.set_postfix(
                loss=f"{val_loss_sum / val_total:.4f}",
                acc=f"{val_correct / val_total:.4f}",
            )

    val_loss = val_loss_sum / val_total
    val_acc  = val_correct  / val_total

    # Output logging
    train_losses.append(train_loss)
    val_losses.append(val_loss)
    train_accs.append(train_acc)
    val_accs.append(val_acc)
    cons_losses.append(train_cons)

    elapsed = time.time() - t0
    print(
        f"Epoch {epoch:02d}/{NUM_EPOCHS}  "
        f"cls_loss={train_loss:.4f}  cons_loss={train_cons:.4f}  "
        f"train_acc={train_acc:.4f}  "
        f"val_loss={val_loss:.4f}  val_acc={val_acc:.4f}  "
        f"({elapsed:.0f}s)"
    )

    # Save best checkpoint
    if val_acc > best_val_acc:
        best_val_acc = val_acc
        best_state   = copy.deepcopy(model.state_dict())
        torch.save(best_state, CHECKPOINT_PATH)
        print(f"   New best val_acc={best_val_acc:.4f} - checkpoint saved.")

print(f"\nTraining complete. Best val_acc: {best_val_acc:.4f}")
```

{"model_id":"264304b529e64f7e9bb7efa507743b02","version_major":2,"version_minor":0}

{"model_id":"f340530f371540c1aa8ce22a091f58e0","version_major":2,"version_minor":0}

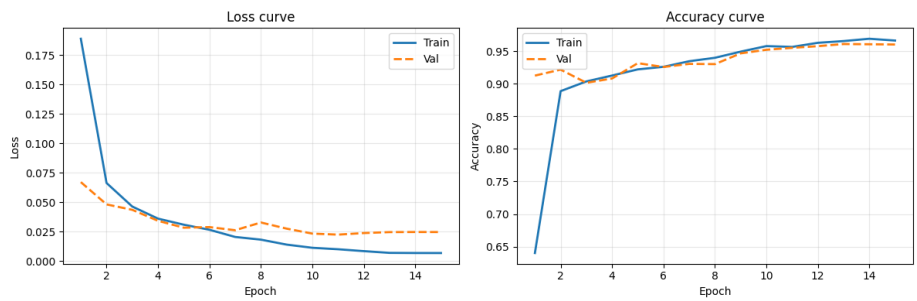
```
Epoch 01/15  cls_loss=0.1889  cons_loss=1.9678  train_acc=0.6401  val_loss=0.0672  val_acc=(
```

```
   New best val_acc=0.9125 - checkpoint saved.
{"model_id":"b4b6d44b541949e08c9a262b26e5630a","version_major":2,"version_minor":0}
{"model_id":"28239095b26347cab141fc5ed589a242","version_major":2,"version_minor":0}
Epoch 02/15  cls_loss=0.0664  cons_loss=1.6835  train_acc=0.8887  val_loss=0.0483  val_acc=0
   New best val_acc=0.9217 - checkpoint saved.
{"model_id":"eba5d0e690584c8fa3e92d754d3a318d","version_major":2,"version_minor":0}
{"model_id":"d408737b64744308b58e7e480ac7b6c0","version_major":2,"version_minor":0}
Epoch 03/15  cls_loss=0.0464  cons_loss=1.2193  train_acc=0.9036  val_loss=0.0437  val_acc=0
{"model_id":"eaa587be9c5e4a6ead91f47aea8013a6","version_major":2,"version_minor":0}
{"model_id":"9d6768ee5bbe4accaf09ed5410910f63","version_major":2,"version_minor":0}
Epoch 04/15  cls_loss=0.0362  cons_loss=0.8997  train_acc=0.9126  val_loss=0.0343  val_acc=0
{"model_id":"025c7578fd37430dbec381eb18f3c0ed","version_major":2,"version_minor":0}
{"model_id":"884e224711af4e3fb1816428b7ae87e3","version_major":2,"version_minor":0}
Epoch 05/15  cls_loss=0.0310  cons_loss=0.6746  train_acc=0.9221  val_loss=0.0285  val_acc=0
   New best val_acc=0.9316 - checkpoint saved.
{"model_id":"de8b992a0d4f4f48b3e26dcab510fe44","version_major":2,"version_minor":0}
{"model_id":"00909c519a3c4a15b0eb157d1fa3ee21","version_major":2,"version_minor":0}
Epoch 06/15  cls_loss=0.0267  cons_loss=0.5204  train_acc=0.9260  val_loss=0.0290  val_acc=0
{"model_id":"fe7db6bc34944d3d8560b244bc5119ba","version_major":2,"version_minor":0}
{"model_id":"48b5ccf379174c51aed489e684404100","version_major":2,"version_minor":0}
Epoch 07/15  cls_loss=0.0206  cons_loss=0.4229  train_acc=0.9346  val_loss=0.0263  val_acc=0
{"model_id":"ce81f144a2804c5598496e45a62bb482","version_major":2,"version_minor":0}
{"model_id":"5ca591acebb54b47be616e8fe6b44faa","version_major":2,"version_minor":0}
Epoch 08/15  cls_loss=0.0183  cons_loss=0.3652  train_acc=0.9399  val_loss=0.0329  val_acc=0
{"model_id":"63a18e6d2aea4a9e979371d19c704eb0","version_major":2,"version_minor":0}
{"model_id":"bb9b954633c047429e31d30ef6be9efb","version_major":2,"version_minor":0}
Epoch 09/15  cls_loss=0.0141  cons_loss=0.3286  train_acc=0.9494  val_loss=0.0277  val_acc=0
   New best val_acc=0.9467 - checkpoint saved.
{"model_id":"b31461ed7ddf41a59f4f625ec291ba81","version_major":2,"version_minor":0}
{"model_id":"9fb9a494ea84439eaada5a658845f186","version_major":2,"version_minor":0}
Epoch 10/15  cls_loss=0.0114  cons_loss=0.2947  train_acc=0.9579  val_loss=0.0235  val_acc=0
   New best val_acc=0.9522 - checkpoint saved.
```

```
{"model_id":"103a1e4e960d4d0d97b98c3fd8c4b3dd","version_major":2,"version_minor":0}

{"model_id":"7d292271ca574d2f9d11691c41ebdd0d","version_major":2,"version_minor":0}

Epoch 11/15  cls_loss=0.0102  cons_loss=0.2663  train_acc=0.9566  val_loss=0.0226  val_acc=0
   New best val_acc=0.9552 - checkpoint saved.

{"model_id":"67c03c424f994944b313001e21a2b96d","version_major":2,"version_minor":0}

{"model_id":"cb57c104b7e24b26bf2207e5a076d51e","version_major":2,"version_minor":0}

Epoch 12/15  cls_loss=0.0086  cons_loss=0.2569  train_acc=0.9629  val_loss=0.0239  val_acc=0
   New best val_acc=0.9577 - checkpoint saved.

{"model_id":"1c8f8d8a48f6464e808caf81b7865a01","version_major":2,"version_minor":0}

{"model_id":"02297171b83344d3b75f7ad76eda3fae","version_major":2,"version_minor":0}

Epoch 13/15  cls_loss=0.0071  cons_loss=0.2490  train_acc=0.9656  val_loss=0.0247  val_acc=0
   New best val_acc=0.9610 - checkpoint saved.

{"model_id":"8286d1c109444115a417a4e0fc4993d1","version_major":2,"version_minor":0}

{"model_id":"fd211257ded54ab39b21a0d8749480d0","version_major":2,"version_minor":0}

Epoch 14/15  cls_loss=0.0070  cons_loss=0.2402  train_acc=0.9691  val_loss=0.0248  val_acc=0

{"model_id":"859d968911314bafb5c082b6423b2276","version_major":2,"version_minor":0}

{"model_id":"4f462d59e25646099daa0a53ddd051d6","version_major":2,"version_minor":0}

Epoch 15/15  cls_loss=0.0070  cons_loss=0.2402  train_acc=0.9664  val_loss=0.0248  val_acc=0

Training complete. Best val_acc: 0.9610
```

```python
from helpers import plot_training_curves

plot_training_curves(
    train_losses, val_losses,
    train_accs,   val_accs,
    save_path=os.path.join(PROJECT_ROOT, "training_curves.png"),
)
```

```
Training curves saved to /content/drive/MyDrive/BME548_Final_Project/training_curves.png
```



6. Evaluation: metrics & confusion matrix

```
# Load best checkpoint before evaluation
model.load_state_dict(torch.load(CHECKPOINT_PATH, map_location=device))
model.eval()
print(f"Loaded best checkpoint from {CHECKPOINT_PATH}")
```

```
Loaded best checkpoint from /content/drive/MyDrive/BME548_Final_Project/best_model.pth
```

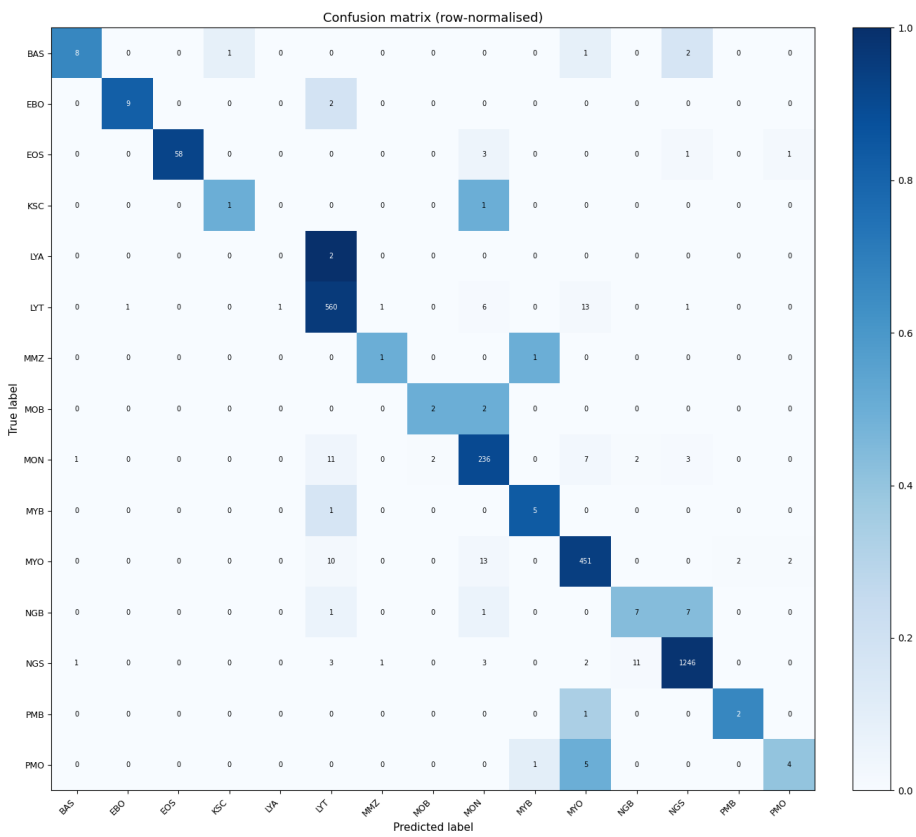
```
from helpers import compute_metrics

print("=== Test set metrics ===")
report = compute_metrics(model, test_loader, class_names, device)

print(f"\nMacro F1        : {report['macro_f1']:.4f}")
print(f"Macro Precision : {report['macro_precision']:.4f}")
print(f"Macro Recall    : {report['macro_recall']:.4f}")
```

```
=== Test set metrics ===

Class   Precision  Recall      F1  Support
-----------------------------------------------
BAS        0.8000  0.6667  0.7273       12
EBO        0.9000  0.8182  0.8571       11
EOS        1.0000  0.9206  0.9587       63
KSC        0.5000  0.5000  0.5000        2
LYA        0.0000  0.0000  0.0000        2
LYT        0.9492  0.9605  0.9548      583
MMZ        0.3333  0.5000  0.4000        2
MOB        0.5000  0.5000  0.5000        4
MON        0.8906  0.9008  0.8956      262
MYB        0.7143  0.8333  0.7692        6
MYO        0.9396  0.9435  0.9415      478
NGB        0.3500  0.4375  0.3889       16
NGS        0.9889  0.9834  0.9861     1267
PMB        0.5000  0.6667  0.5714        3
PMO        0.5714  0.4000  0.4706       10
```

```
----------------------------------------------
Macro      0.6625  0.6687  0.6614

Macro F1        : 0.6614
Macro Precision : 0.6625
Macro Recall    : 0.6687
```

```python
from helpers import plot_confusion_matrix

plot_confusion_matrix(
    report["all_labels"],
    report["all_preds"],
    class_names,
    save_path=os.path.join(PROJECT_ROOT, "confusion_matrix.png"),
)
```

```
Confusion matrix saved to /content/drive/MyDrive/BME548_Final_Project/confusion_matrix.png
```



7. Robustness curves

```
# Robustness curves sweep one perturbation parameter at a time.
# We compare the proposed model (Lab correction) against a baseline
# (same pipeline but with correction module weights reset to identity).

from preprocess import parse_annotations, resolve_image_paths
from preprocess import IMAGE_FOLDER, SPLIT_RATIOS, SEED
from sklearn.model_selection import train_test_split

# Reconstruct test_df for the robustness curve helper
df_all = parse_annotations(os.path.join(LOCAL_DATA_ROOT, "annotations.dat"))
df_all = resolve_image_paths(df_all, LOCAL_DATA_ROOT, IMAGE_FOLDER)
_, test_df = train_test_split(
    df_all, test_size=SPLIT_RATIOS[2],
    stratify=df_all["label"], random_state=SEED
)
test_df = test_df.reset_index(drop=True)
print(f"Test split size for robustness curves: {len(test_df)}")
```

```
Test split size for robustness curves: 2721
```

```
# Build a baseline model (same architecture, just correction module reset to identity)
from models import build_model

baseline_model, _ = build_model(
    num_classes   = NUM_CLASSES,
    base_channels = BASE_CHANNELS,
    num_blocks    = NUM_BLOCKS,
    dropout       = DROPOUT,
    pretrained    = True,
)
# Load best backbone weights but reinitialise only the correction module
# to simulate a pipeline without any learned stain correction
baseline_state = torch.load(CHECKPOINT_PATH, map_location=device)
baseline_model.load_state_dict(baseline_state)

# Reset correction module weights to near-identity
from models import CorrectionModule
baseline_model.correction = CorrectionModule(
    base_channels=BASE_CHANNELS, num_blocks=NUM_BLOCKS
).to(device)
baseline_model.eval()
print("Baseline model ready (correction module reset to identity).")
```

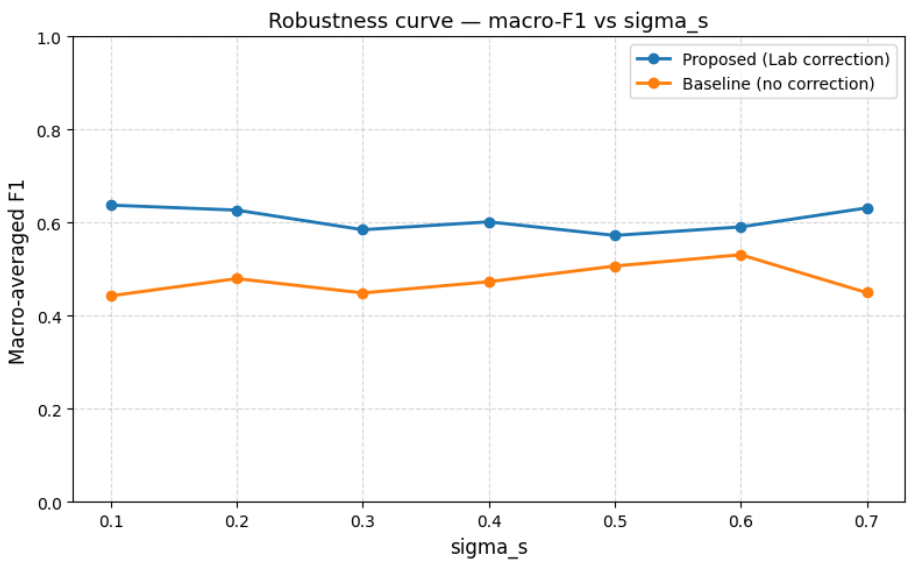
```
GPU available: NVIDIA RTX PRO 6000 Blackwell Server Edition -
training on CUDA.
```

```
Model summary:
  Correction module params :     55,872
  ConvNeXt-Tiny params     : 27,831,663
  Total params             : 27,887,535
  Trainable params         : 27,887,535
Baseline model ready (correction module reset to identity).
```

```python
import importlib
import helpers
importlib.reload(helpers)
from helpers import robustness_curve
import os

models_dict = {
    "Proposed (Lab correction)": model,
    "Baseline (no correction)" : baseline_model,
}

#SIGMA_VALUES = [0.0, 0.02, 0.05, 0.10, 0.15, 0.20]
SIGMA_VALUES = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7]

# Sweep sigma_s (stain spectrum variability)
print("\n--- Sweeping sigma_s (stain spectrum) ---")
results_s = robustness_curve(
    models_dict, test_df, class_names, device,
    param_name   = "sigma_s",
    param_values = SIGMA_VALUES,
    save_path    = os.path.join(PROJECT_ROOT, "extreme_robustness_sigma_s.png"),
)

# Sweep sigma_c (concentration variability)
print("\n--- Sweeping sigma_c (concentration) ---")
results_c = robustness_curve(
    models_dict, test_df, class_names, device,
    param_name   = "sigma_c",
    param_values = SIGMA_VALUES,
    save_path    = os.path.join(PROJECT_ROOT, "extereme_robustness_sigma_c.png"),
)

# Sweep sigma_w (white balance / illuminant variability)
print("\n--- Sweeping sigma_w (white balance) ---")
results_w = robustness_curve(
    models_dict, test_df, class_names, device,
    param_name   = "sigma_w",
    param_values = SIGMA_VALUES,
```

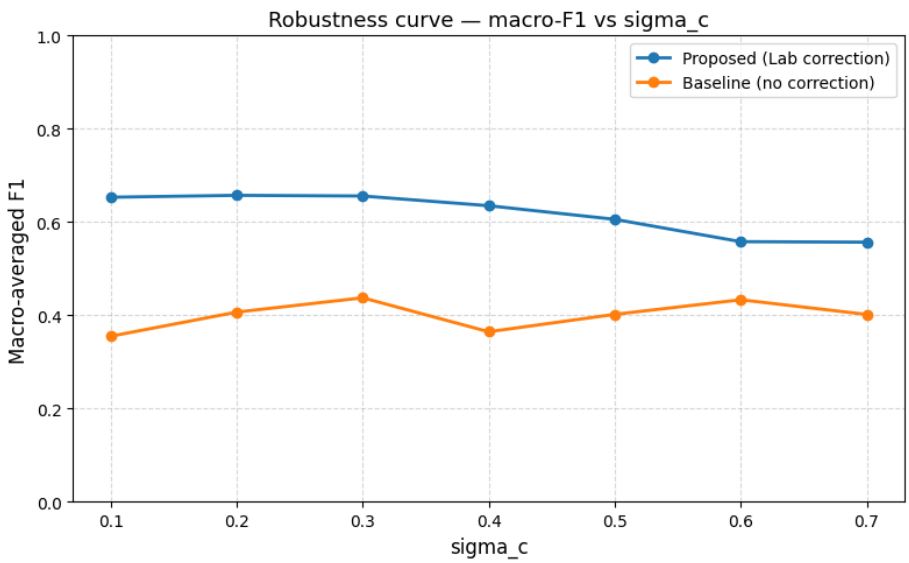
```
    save_path    = os.path.join(PROJECT_ROOT, "extereme_robustness_sigma_w.png"),
)
```

```
--- Sweeping sigma_s (stain spectrum) ---
  Proposed (Lab correction)  sigma_s=0.10  macro-F1=0.6377
  Baseline (no correction)  sigma_s=0.10  macro-F1=0.4429
  Proposed (Lab correction)  sigma_s=0.20  macro-F1=0.6270
  Baseline (no correction)  sigma_s=0.20  macro-F1=0.4799
  Proposed (Lab correction)  sigma_s=0.30  macro-F1=0.5849
  Baseline (no correction)  sigma_s=0.30  macro-F1=0.4491
  Proposed (Lab correction)  sigma_s=0.40  macro-F1=0.6018
  Baseline (no correction)  sigma_s=0.40  macro-F1=0.4732
  Proposed (Lab correction)  sigma_s=0.50  macro-F1=0.5726
  Baseline (no correction)  sigma_s=0.50  macro-F1=0.5069
  Proposed (Lab correction)  sigma_s=0.60  macro-F1=0.5909
  Baseline (no correction)  sigma_s=0.60  macro-F1=0.5312
  Proposed (Lab correction)  sigma_s=0.70  macro-F1=0.6317
  Baseline (no correction)  sigma_s=0.70  macro-F1=0.4498
Robustness curve saved to /content/drive/MyDrive/BME548_Final_Project/extreme_robustness_sig
```



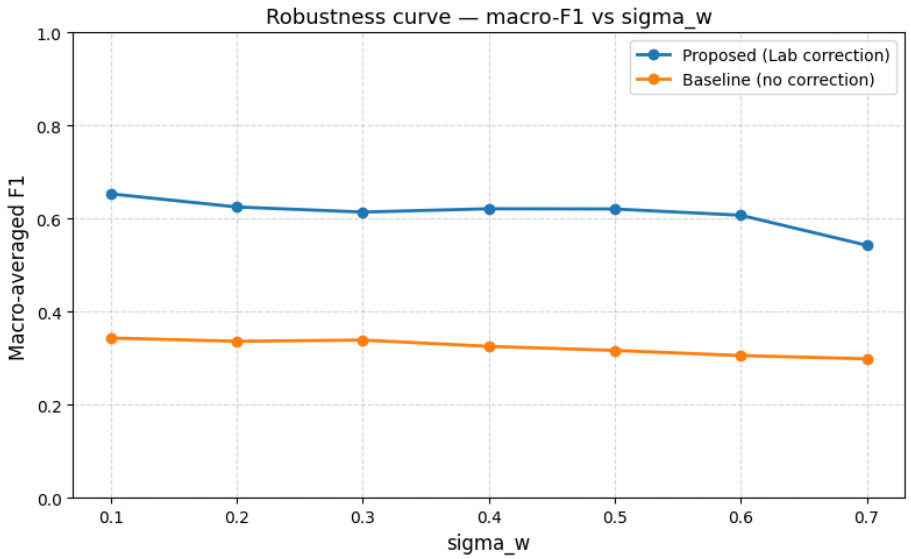

```
--- Sweeping sigma_c (concentration) ---
  Proposed (Lab correction)  sigma_c=0.10  macro-F1=0.6533
  Baseline (no correction)  sigma_c=0.10  macro-F1=0.3553
  Proposed (Lab correction)  sigma_c=0.20  macro-F1=0.6574
  Baseline (no correction)  sigma_c=0.20  macro-F1=0.4070
  Proposed (Lab correction)  sigma_c=0.30  macro-F1=0.6559
```

```
  Baseline (no correction)  sigma_c=0.30  macro-F1=0.4376
  Proposed (Lab correction)  sigma_c=0.40  macro-F1=0.6351
  Baseline (no correction)  sigma_c=0.40  macro-F1=0.3649
  Proposed (Lab correction)  sigma_c=0.50  macro-F1=0.6059
  Baseline (no correction)  sigma_c=0.50  macro-F1=0.4022
  Proposed (Lab correction)  sigma_c=0.60  macro-F1=0.5580
  Baseline (no correction)  sigma_c=0.60  macro-F1=0.4334
  Proposed (Lab correction)  sigma_c=0.70  macro-F1=0.5569
  Baseline (no correction)  sigma_c=0.70  macro-F1=0.4020
Robustness curve saved to /content/drive/MyDrive/BME548_Final_Project/extereme_robustness_si
```




```
--- Sweeping sigma_w (white balance) ---
  Proposed (Lab correction)  sigma_w=0.10  macro-F1=0.6533
  Baseline (no correction)  sigma_w=0.10  macro-F1=0.3437
  Proposed (Lab correction)  sigma_w=0.20  macro-F1=0.6252
  Baseline (no correction)  sigma_w=0.20  macro-F1=0.3367
  Proposed (Lab correction)  sigma_w=0.30  macro-F1=0.6142
  Baseline (no correction)  sigma_w=0.30  macro-F1=0.3393
  Proposed (Lab correction)  sigma_w=0.40  macro-F1=0.6213
  Baseline (no correction)  sigma_w=0.40  macro-F1=0.3258
  Proposed (Lab correction)  sigma_w=0.50  macro-F1=0.6209
  Baseline (no correction)  sigma_w=0.50  macro-F1=0.3171
  Proposed (Lab correction)  sigma_w=0.60  macro-F1=0.6074
  Baseline (no correction)  sigma_w=0.60  macro-F1=0.3060
  Proposed (Lab correction)  sigma_w=0.70  macro-F1=0.5429
  Baseline (no correction)  sigma_w=0.70  macro-F1=0.2991
```

```
Robustness curve saved to /content/drive/MyDrive/BME548_Final_Project/extereme_robustness_si
```



8. Grad-CAM visualisation

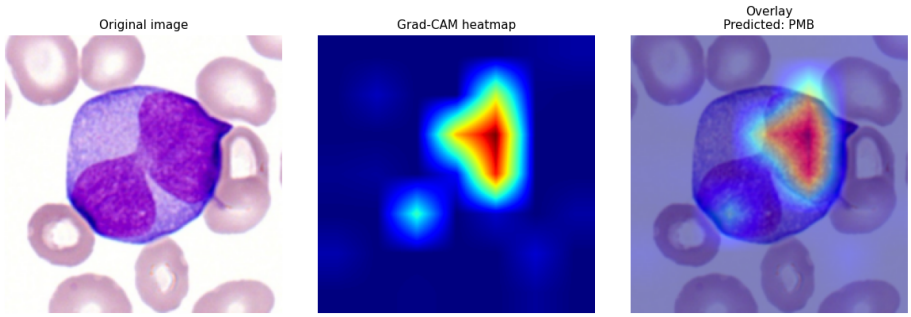
```python
from helpers import plot_gradcam

# Show Grad-CAM for one example from each class
# We iterate through the test loader and collect the first image per class

seen_classes = set()
class_examples = {}

for imgs, labels in test_loader:
    for img, label in zip(imgs, labels):
        cls = label.item()
        if cls not in seen_classes:
            class_examples[cls] = img.unsqueeze(0)  # (1, 3, H, W)
            seen_classes.add(cls)
    if len(seen_classes) == NUM_CLASSES:
        break

print(f"Collected one example for each of the {len(class_examples)} classes.")
```

```
Collected one example for each of the 15 classes.
```

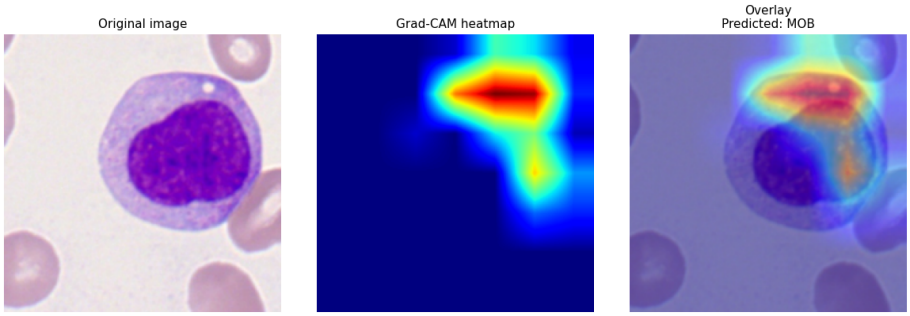
```python
# Plot Grad-CAM for a selection of clinically important classes
# MYO = Myeloblast (blast cell)
# NGS = Neutrophil segmented (dominant class)
```

```python
# LYT = Lymphocyte typical
# MON = Monocyte

import importlib
import helpers
importlib.reload(helpers)
from helpers import plot_gradcam
import os

CLASSES_TO_VISUALISE = ["PMB", "MOB", "MYB", "LYA", "KSC", "MMZ"]

for cls_name in CLASSES_TO_VISUALISE:
    cls_idx = class_to_idx[cls_name]
    if cls_idx not in class_examples:
        print(f"No example found for {cls_name}, skipping.")
        continue

    print(f"\nGrad-CAM for class: {cls_name} ({abbrev_map.get(cls_name, cls_name)})")
    plot_gradcam(
        model,
        class_examples[cls_idx],
        class_names,
        device,
        class_idx = cls_idx,
        save_path = os.path.join(PROJECT_ROOT, f"rare_gradcam_{cls_name}.png"),
    )
```

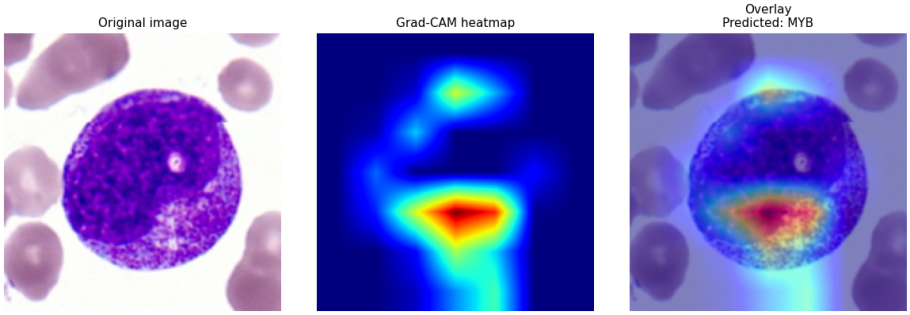
```
Grad-CAM for class: PMB (Promyelocyte (bilobled))
Grad-CAM saved to /content/drive/MyDrive/BME548_Final_Project/rare_gradcam_PMB.png
```



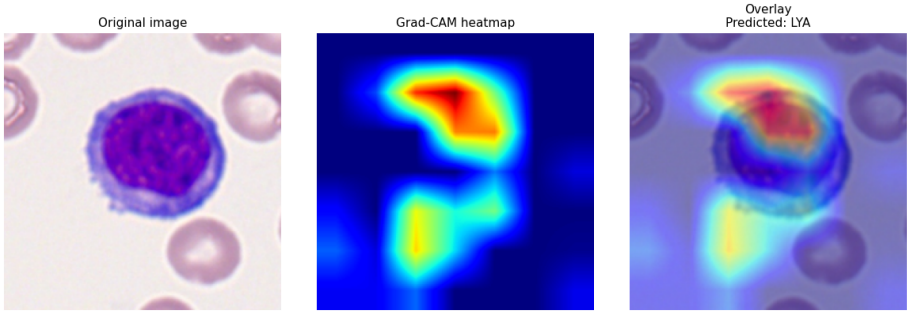

```
Grad-CAM for class: MOB (Monoblast)
Grad-CAM saved to /content/drive/MyDrive/BME548_Final_Project/rare_gradcam_MOB.png
```



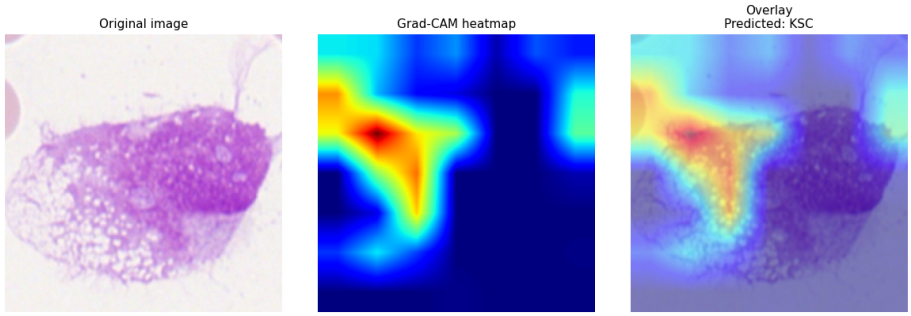

```
Grad-CAM for class: MYB (Myelocyte)
Grad-CAM saved to /content/drive/MyDrive/BME548_Final_Project/rare_gradcam_MYB.png
```



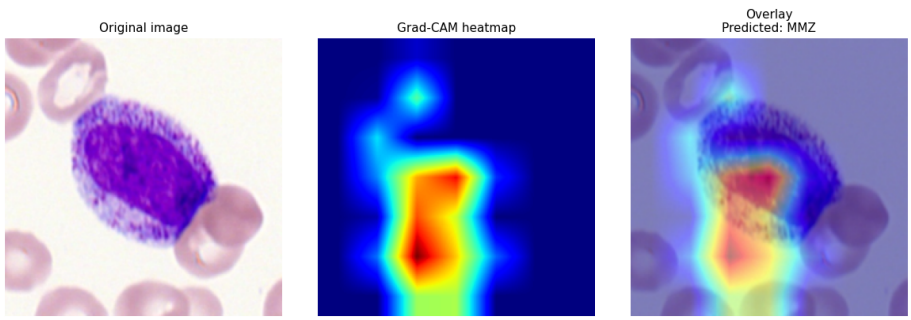

```
Grad-CAM for class: LYA (Lymphocyte (atypical))
Grad-CAM saved to /content/drive/MyDrive/BME548_Final_Project/rare_gradcam_LYA.png
```




```
Grad-CAM for class: KSC (Smudge cell)
Grad-CAM saved to /content/drive/MyDrive/BME548_Final_Project/rare_gradcam_KSC.png
```



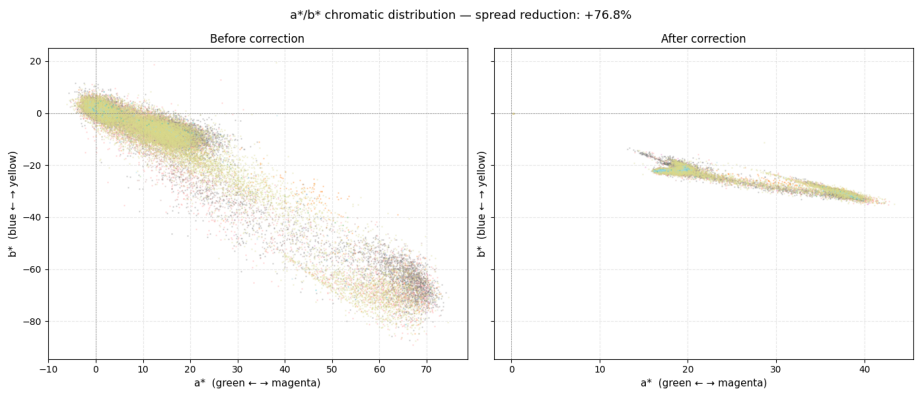

```
Grad-CAM for class: MMZ (Metamyelocyte)
Grad-CAM saved to /content/drive/MyDrive/BME548_Final_Project/rare_gradcam_MMZ.png
```



9. a/*b* chromatic distribution diagnostic

```
from helpers import plot_lab_distributions

# If the correction module is working as intended, the a*/b* distributions
# should be tighter, indicating that stain-induced chromatic variability has been normalised

print("Plotting a*/b* chromatic distributions (test set)...")
plot_lab_distributions(
    model,
    test_loader,
    device,
    n_batches = 20,
    save_path = os.path.join(PROJECT_ROOT, "extreme_lab_distributions.png"),
)
```

```
Plotting a*/b* chromatic distributions (test set)...
Lab distribution plot saved to /content/drive/MyDrive/BME548_Final_Project/extreme_lab_distr
```




```
a* std before: 18.152  |  after: 5.930
b* std before: 19.936  |  after: 2.895
Combined spread reduction: +76.8%
```